

Neural Networks

Layers, training and generalization

Theory 05

22 September 2026

Tiago Fonseca

A two-hour online class with worked examples and discussion



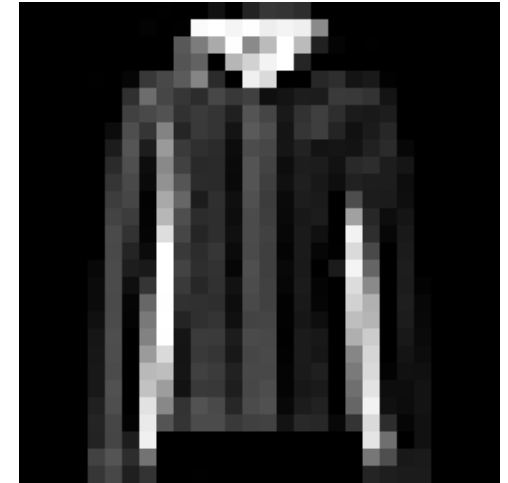
Which examples are harder to distinguish?



A



B



C

Chat: choose the most confusing pair and explain one visual clue.

Real 28×28 Fashion-MNIST images. Labels discussed before moving on.

Today's route

01 **Layers and a complete prediction**

What the model computes, and what each parameter does.

02 **Training and parameter updates**

Why loss decreases and how gradients guide learning.

03 **Generalization and model diagnosis**

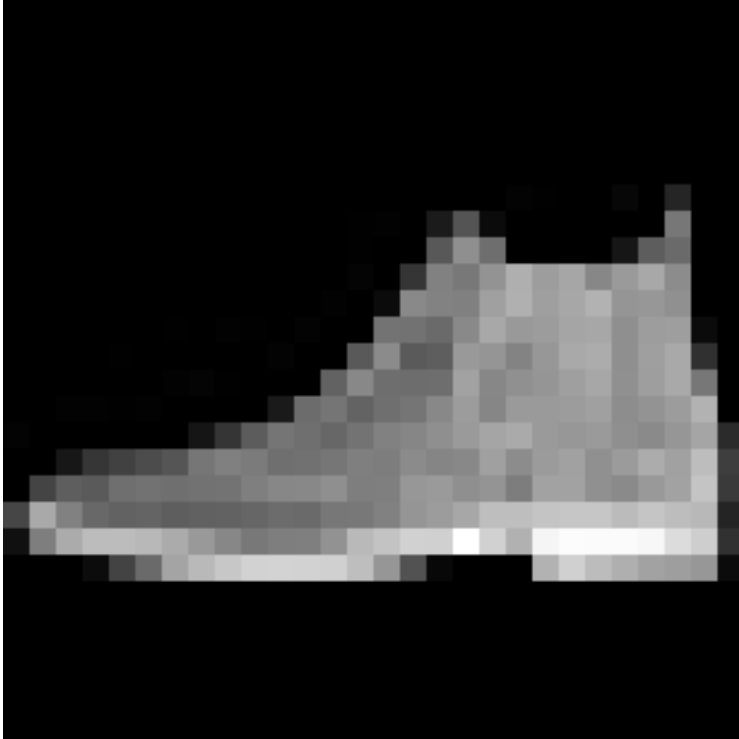
Learning curves, model selection and informative errors.

Layers and forward pass

01

How do 784 pixel values become a class prediction?

One image, 784 measurements



$$28 \times 28 = 784 \text{ pixels}$$

Each pixel records an intensity.
The label supplies the target class.

PL2 scales intensities from 0–255
to the range 0–1.

The image carries input values. The class label is separate training information.

Flatten preserves the values

Small image

0.0 0.8
0.3 1.0

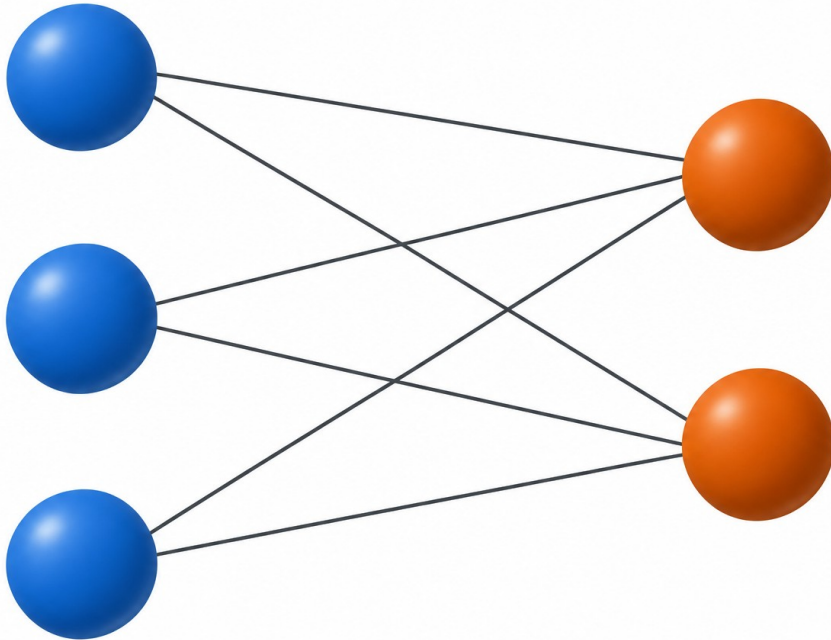
Flattened input

[0.0, 0.8, 0.3, 1.0]

The same values appear in a fixed order.

A Dense layer receives a vector. Pixel positions remain identifiable by index, but the layer has no built-in rule for neighbouring pixels.

A Dense layer computes several neurons



Every output uses every input.

Each neuron has its own weights and bias.

The layer evaluates all neurons for the same example.

Shown: 3 inputs and 2 output neurons. Connections represent weights.

Weights and biases in a Dense layer

n inputs × m outputs + m biases

Example

Weights

Biases

Total

3 inputs, 2 outputs

$3 \times 2 = 6$

2

8

A parameter is one trainable number.

This count assumes a standard Dense layer with a bias for every output.

Matrix notation describes the whole layer

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

Quantity	Shape	Meaning
$\mathbf{x}$	784×1	One flattened image
$\mathbf{W}$	64×784	One row per hidden neuron
$\mathbf{b}$ and $\mathbf{z}$	64×1	64 biases and 64 pre-activations

Column-vector convention. Keras stores the Dense kernel as $\text{input_dim} \times \text{units}$.

The PL2 baseline

Flatten

784 input values

Dense(10) + Softmax

10 class scores

Each class receives a weighted sum of the original pixels.

There is no hidden layer. The decision boundaries are linear in the pixel input space.

Baseline parameter count

7,840

weights: 784×10

+ 10

biases

7,850 trainable parameters

Flatten and Softmax add no trainable parameters in this model.

The PL2 multilayer perceptron



784 inputs

64 hidden units + ReLU

10 outputs + Softmax

The hidden layer learns an intermediate representation before classification.

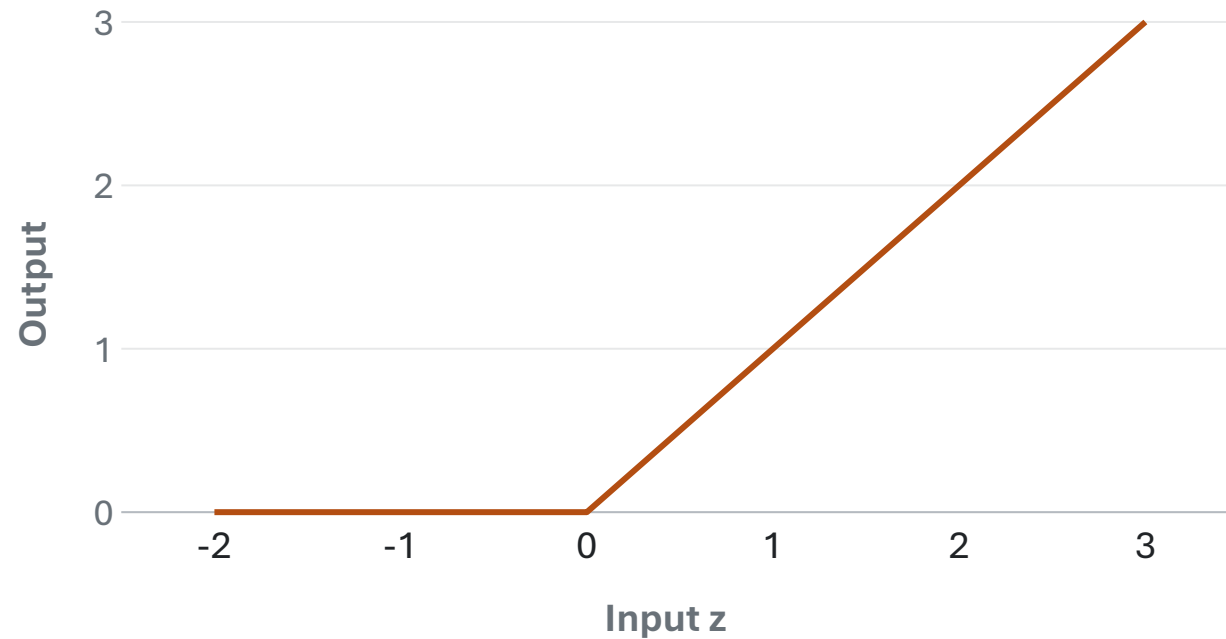
What can a hidden unit represent?

A hidden activation is a learned response to the input.

A unit may respond to a useful combination of pixel intensities.
The output layer combines many of these responses.

“Unit 7 = sleeve detector” is only a possible interpretation.
Useful features may be distributed across several units.

ReLU introduces a nonlinear step



$$\text{ReLU}(z) = \max(0, z)$$

Negative values become zero.
Positive values pass through.

This bend allows successive layers to represent nonlinear relationships.

Stacking linear layers is still linear

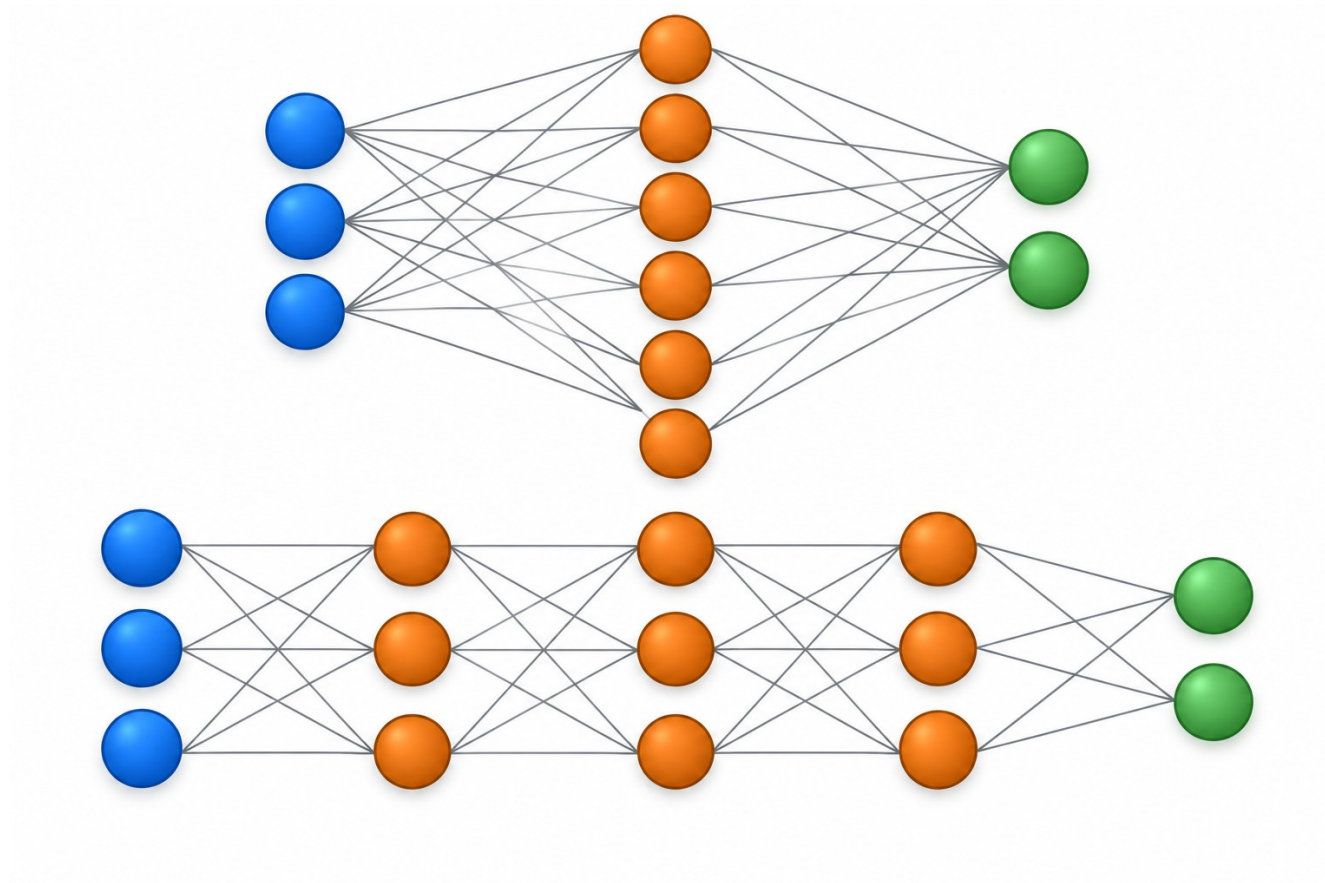
First layer: $h = 2x + 1$

Second layer: $y = 3h - 4$

Together: $y = 3(2x + 1) - 4 = 6x - 1$

A nonlinear activation between layers changes what the network can represent.

Width and depth are different choices



Width

Units in a hidden layer.

Depth

Successive hidden transformations.

The cost of different architectures

Architecture after Flatten	Parameter calculation	Total
Dense(10)	$784 \times 10 + 10$	7,850
Dense(64), Dense(10)	$(784 \times 64 + 64) + (64 \times 10 + 10)$	50,890
Dense(32), Dense(32), Dense(10)	$25,120 + 1,056 + 330$	26,506

The deeper example has fewer parameters than the wider MLP.

Hidden layers use ReLU. The output uses Softmax. Parameter counts include biases.

Chat challenge: a wider hidden layer

Flatten, Dense(128, ReLU), Dense(10, Softmax)

How many trainable parameters?

A 100,480

B 101,770

Take 45 seconds. Include the output layer and all biases.

The output layer also has parameters

Layer	Weights	Biases	Parameters
784 to 128	100,352	128	100,480
128 to 10	1,280	10	1,290

$$\text{B: } 100,480 + 1,290 = 101,770$$

Forward pass: values change, weights stay fixed

Stage

Values produced

Flatten

784 pixel intensities

Dense + ReLU

64 hidden activations

Dense

10 logits

Softmax

10 normalized class scores

A logit is a raw score before Softmax.

A tiny network we can calculate by hand

Two inputs

$$x_1 = 0.8 \quad x_2 = 0.3$$

Hidden neuron	Weight for x_1	Weight for x_2	Bias
1	1.2	-0.5	0.1
2	-0.5	0.8	-0.1

Hidden values and ReLU

Neuron 1

$$\begin{aligned} z_1 &= 1.2 \times 0.8 - 0.5 \times 0.3 + 0.1 \\ &= 0.91 \end{aligned}$$

$$h_1 = \text{ReLU}(0.91) = 0.91$$

Neuron 2

$$z_2 = -0.5 \times 0.8 + 0.8 \times 0.3 - 0.1 = -0.26$$

$$h_2 = \text{ReLU}(-0.26) = 0$$

The output neuron combines hidden values

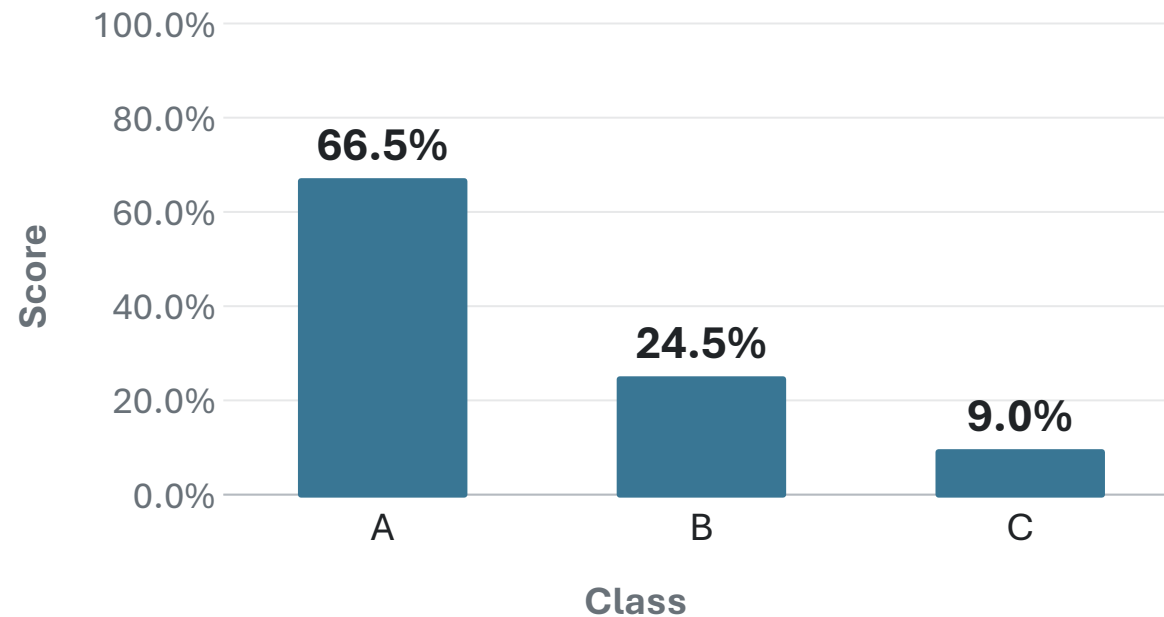
$$h_1 = 0.91 \quad h_2 = 0$$

$$z = 1.4h_1 - 0.7h_2 - 0.2$$

$$z = 1.4 \times 0.91 - 0.7 \times 0 - 0.2 = 1.074$$

1.074 is a logit. A multiclass model computes one logit per class.

Softmax turns logits into class scores



Logits: [2, 1, 0]

$$p_i = \exp(z_i) / \sum_j \exp(z_j)$$

Scores $\approx$ [0.665, 0.245, 0.090]

Illustrative three-class example. Scores sum to 1. The largest score selects class A.

Inference uses the learned function

A new image changes the activations.

The stored weights and biases remain fixed during ordinary inference.

Training adds a target label, a loss calculation and a parameter update.

Training

02

How do we choose useful values for all those parameters?

One training update

1 Predict

Compute outputs using the current parameters.

2 Measure loss

Compare the outputs with the target labels.

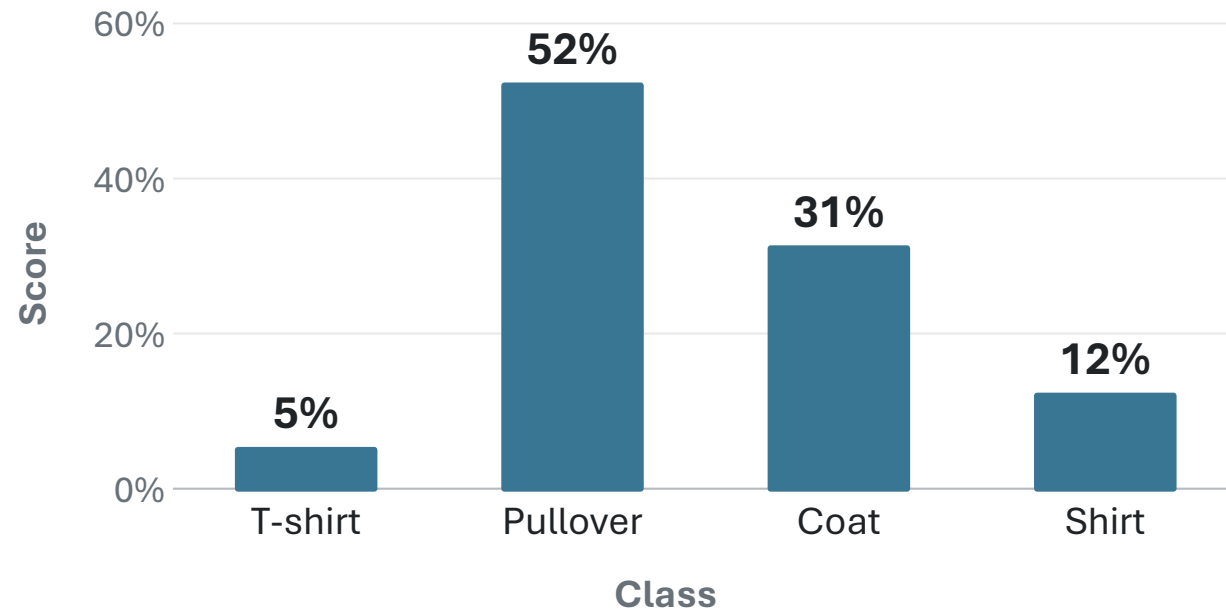
3 Compute gradients

Compute local sensitivity of loss to each parameter.

4 Update parameters

Apply the optimizer rule, then use the next batch.

A prediction and its target



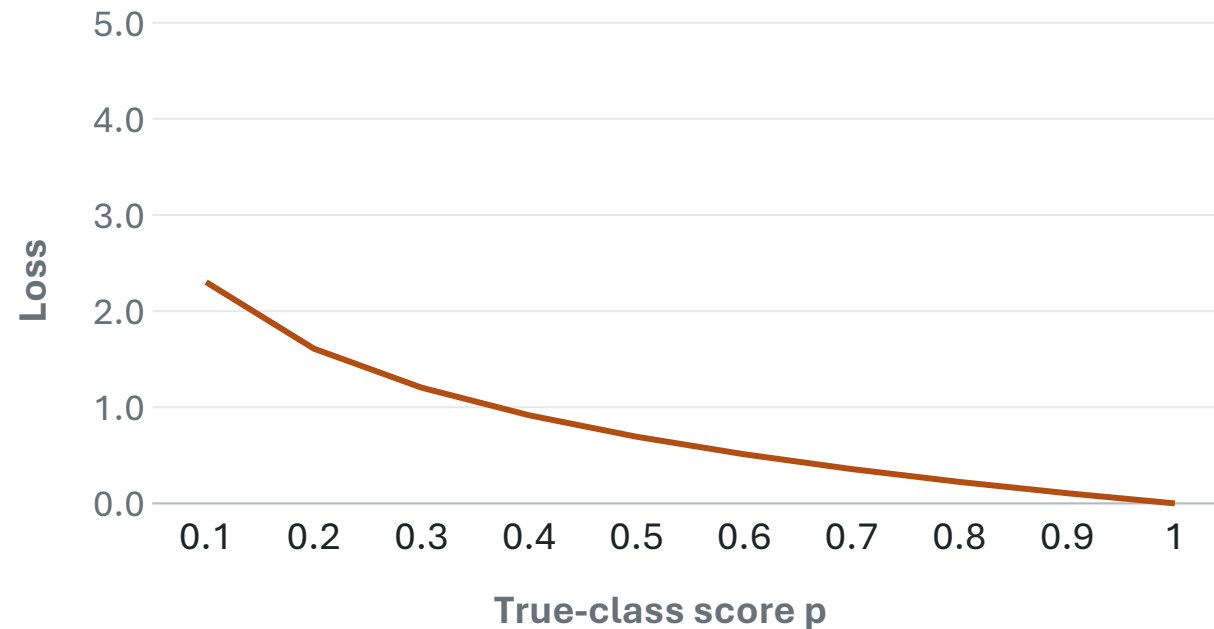
True class: Coat

**Predicted class:
Pullover**

The true-class score is 0.31.

Illustrative four-class example. We need a loss that improves the true-class score.

Cross-entropy measures the prediction error



$$L = -\ln(p_{\text{true}})$$

$$p = 0.90 \quad L \approx 0.105$$

$$p = 0.31 \quad L \approx 1.171$$

$$p = 0.01 \quad L \approx 4.605$$

One labelled example with a single correct class. Batch loss usually averages example losses.

Accuracy and loss answer different questions

Binary example	True-class score	Correct?	Loss
Prediction A	0.51	Yes	0.673
Prediction B	0.99	Yes	0.010

Both count equally for accuracy. Their losses are very different.

Same true class, two alternative predictions. Loss values use $-\ln(p_{\text{true}})$.

A gradient describes the local slope



**If a parameter changes a little,
how does the loss change?**

The gradient describes this local sensitivity.

Landscape analogy. A real network has many parameter dimensions and a more complex loss surface.

Gradient descent updates parameters

$$\text{new value} = \text{old value} - \text{learning rate} \times \text{gradient}$$

Positive gradient: a small increase raises loss.
The update moves the parameter down.

Negative gradient: a small increase lowers loss.
The update moves the parameter up.

One weight before learning

Model: prediction = $w \times x$

$x = 2$ $\text{target} = 1$ $w = 0.2$

Prediction = $0.2 \times 2 = 0.4$

$L = (\text{prediction} - \text{target})^2 = (0.4 - 1)^2 = 0.36$

Toy regression example with squared error, used here to make the update easy to follow.

The gradient tells us the direction

$$L = (2w - 1)^2$$

$$dL/dw = 2(2w - 1) \times 2$$

$$\text{At } w = 0.2: dL/dw = 2(-0.6) \times 2 = -2.4$$

The gradient is negative. Gradient descent increases w .

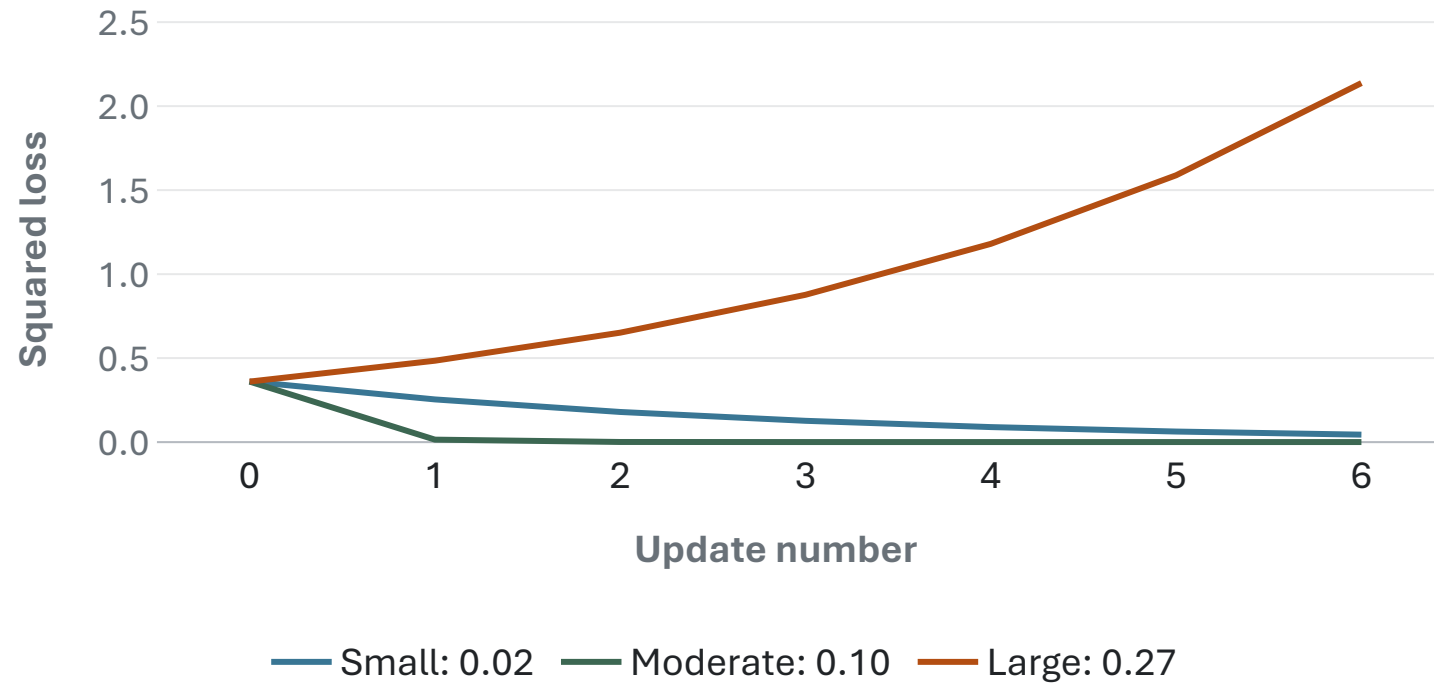
One update reduces this example's loss

Learning rate = 0.1

$$w_{\text{new}} = 0.2 - 0.1 \times (-2.4) = 0.44$$

	Before	After one update
Weight	0.2	0.44
Prediction	0.4	0.88
Squared loss	0.36	0.0144

The learning rate changes the trajectory



Same model.
Same starting weight.
Different step sizes.

A large step can overshoot and increase loss.

Exact trajectories for the previous one-weight example. These rates are not recommendations for PL2.

Backpropagation computes many gradients efficiently

Forward pass

Store intermediate values while calculating the prediction and loss.

Backward pass

Work backwards through the operations to compute parameter gradients.

The optimizer then uses those gradients to update the parameters.

The chain rule connects local sensitivities

weight w

hidden value h

final loss L

$$dL/dw = (dL/dh) \times (dh/dw)$$

If $dL/dh = 3$ and $dh/dw = 2$, then $dL/dw = 6$.

A single-path scalar example. When paths branch, their contributions add.

Adam uses a history of gradients

Gradient descent uses the current gradient and a step size.

Adam tracks moving averages of gradients and squared gradients.

These averages adapt updates for each parameter.
The learning rate still matters.

Chat check: who does what?

Operation

Possible roles

Forward pass

A Compute gradients

Backpropagation

B Change parameter values

Optimizer step

C Compute the prediction

Chat: send the three letters in the order shown.

Five-minute break

5 min

After the break: batches, learning curves
and choosing a model.

A batch groups examples for one update



Batch size = 128

Compute predictions and a mean loss for this group.

Backpropagate that loss, then perform one update.

An epoch contains many updates

54,000 training images Batch size: 128

421 full batches + 1 final batch of 112 images

422 updates per epoch

Assumes every example is used once and the incomplete final batch is kept.

Shuffling changes the batches

Epoch Illustrative sample order

1 A B C D E F G H

2 F A H C B G D E

The same examples appear in different groups and positions.

Random initialization, shuffle order and runtime details can change the training trajectory.

Parameters and hyperparameters

Parameters learned in training

Weights and biases

Training updates their values

They define the fitted function

Choices made around training

Width and depth

Learning rate and batch size

Epoch budget and regularization

Validation helps compare these choices. Ordinary training keeps the architecture fixed.

Generalization

03

Does the learned function work on examples it has not trained on?

Learning useful patterns for new examples



Different shape, texture and details. The class can stay the same.

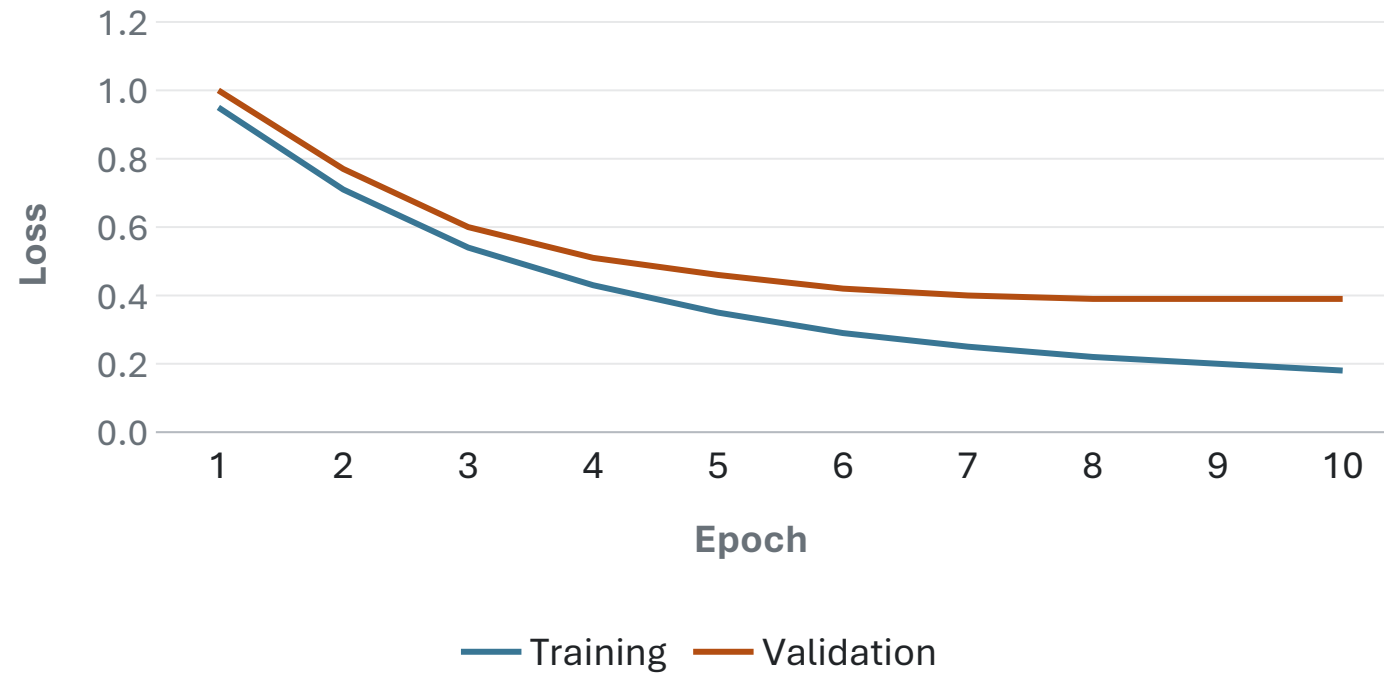
Conceptual illustration of within-class variation, not Fashion-MNIST test evidence.

Training, validation and test have different roles

Split in PL2	Images	Role
Training	54,000	Update the weights
Validation	6,000	Choose the model and stopping point
Test	10,000	Evaluate the selected model

Validation and test predictions use fixed parameters.

Reading a learning curve

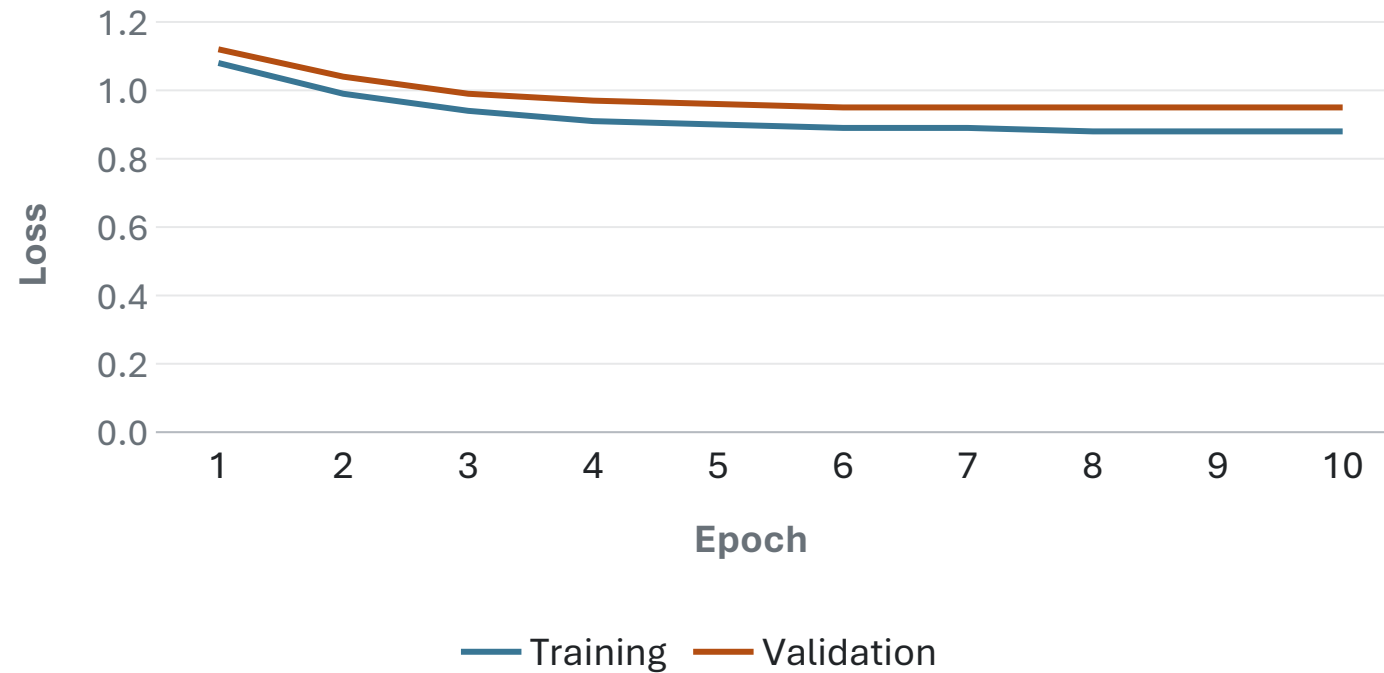


Horizontal axis: epochs.
Vertical axis: loss.

Read the direction, the gap and where improvement stops.

Schematic values for teaching. These curves are not measured PL2 results.

Underfitting: both sets remain poorly fitted

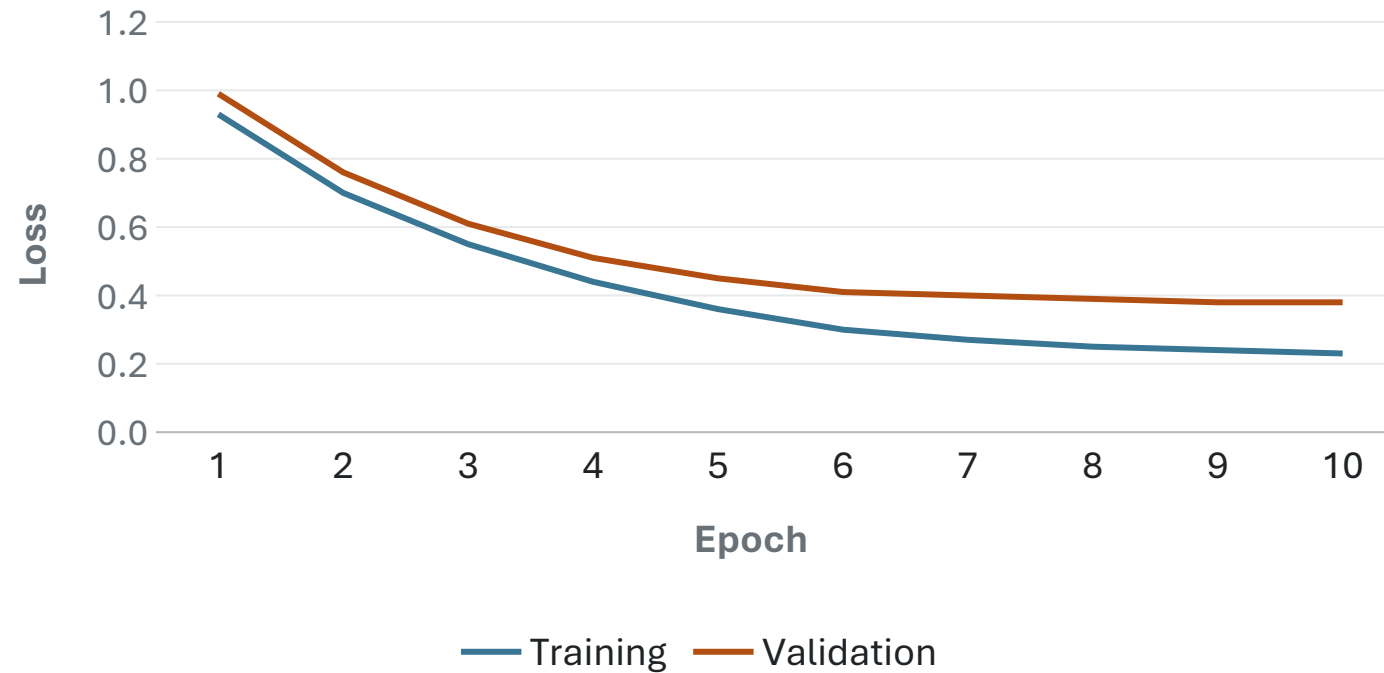


Training loss stays high.
Validation loss stays high.

Check data and
optimization.
Then consider more
capacity.

Schematic example. Low capacity is one possible cause, not the only explanation.

A useful fit improves both sets

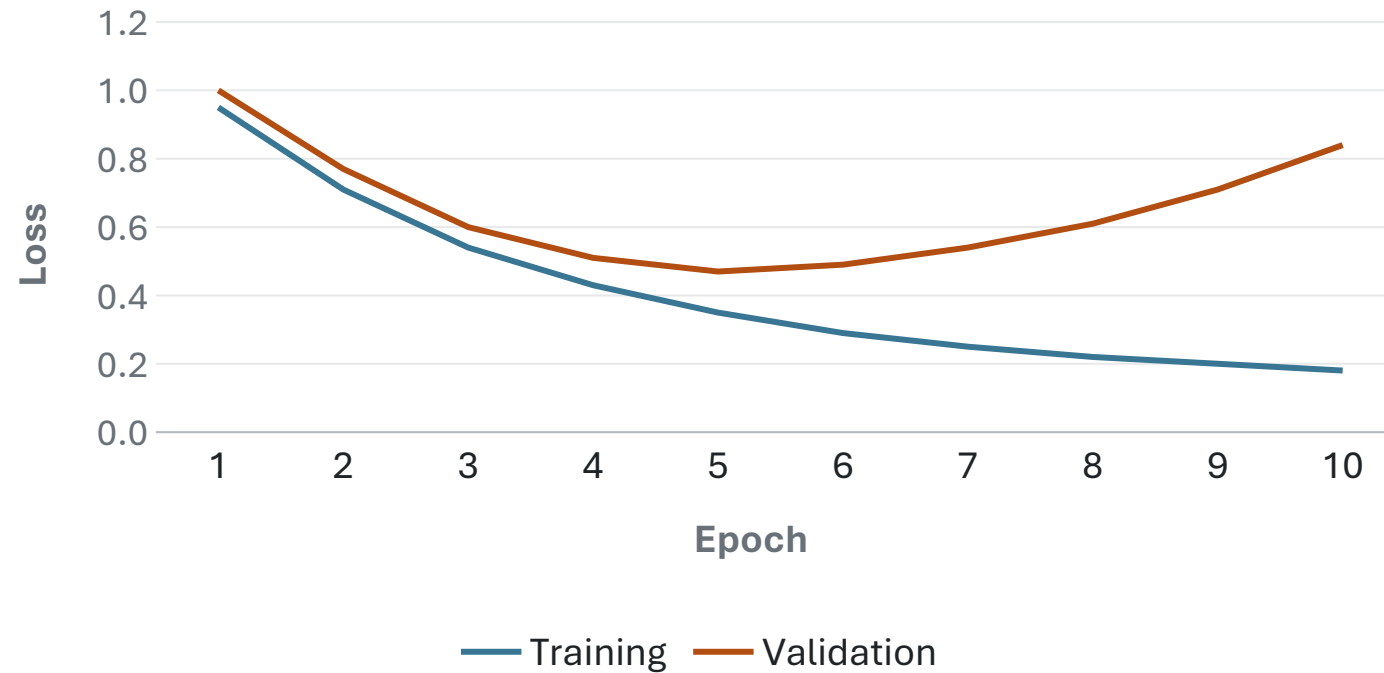


Both losses decrease and then stabilize.

A modest gap is compatible with useful generalization.

Schematic example. The required accuracy and error profile still depend on the task.

Chat diagnosis: should training continue?



A Continue to epoch 50

B Keep an earlier model

Which curve supports your choice?

Schematic values. Assume the goal is to minimize validation loss.

Overfitting and early stopping

Training fit improves while validation performance gets worse.

Epoch	Validation loss	Interpretation
4	0.51	Still improving
5	0.47	Best observed value
6	0.49	No improvement
7	0.54	Second epoch without improvement

With patience = 2, stop after epoch 7 and restore epoch 5.

More capacity can improve training and hurt validation

Model	Parameters	Train accuracy	Val. accuracy
A: one hidden layer of 64	50,890	94.0%	88.0%
B: one hidden layer of 128	101,770	99.0%	87.0%

B fits the training set better. A is the stronger validation candidate.

Hypothetical comparison, not PL2 measurements. Repeat runs if differences are small.

Regularization discourages overly specific solutions

L2 weight regularization

Add a penalty for large squared weights to the training objective.

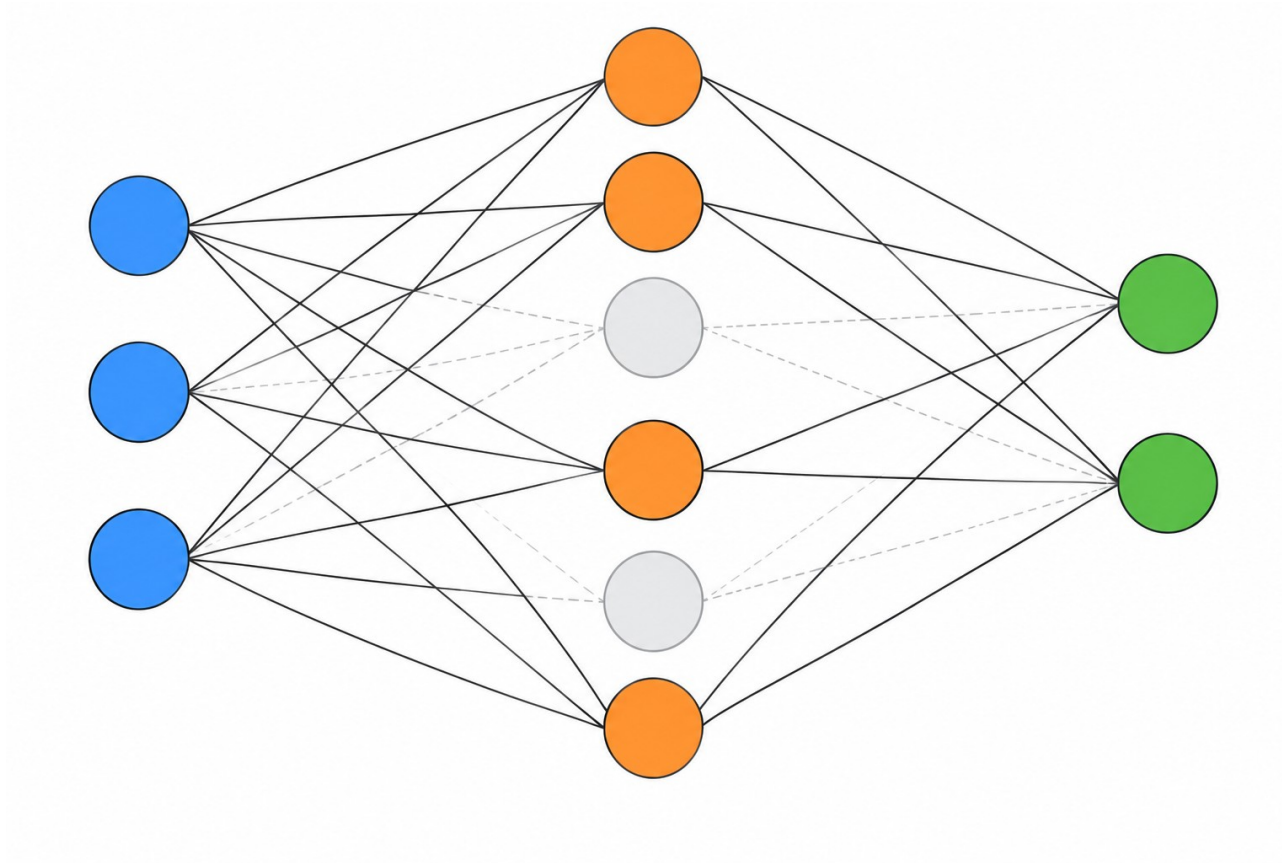
Data loss + $\lambda \times$ sum of squared weights

Strength matters

Too little may have little effect.
Too much can cause underfitting.

Validation guides the choice of λ .

Dropout changes training behaviour



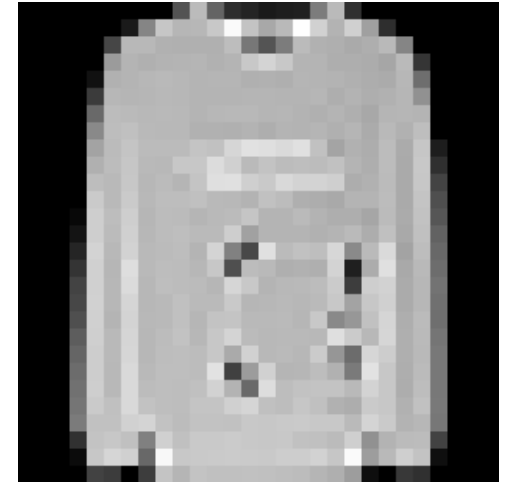
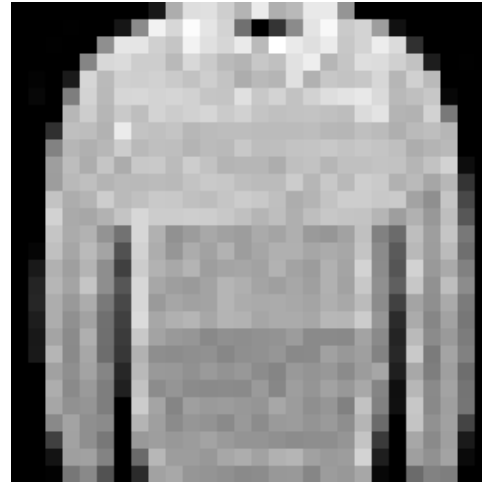
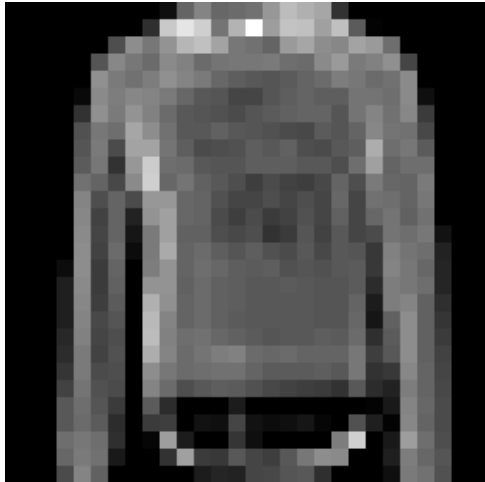
During training

Randomly set some activations to zero. The mask changes.

During inference

Use the full network, with no random dropping.

More useful data can matter more than more layers



Variation within the same class

Training data should cover the variation we expect at use time.

Augmentation must preserve the correct label and plausible inputs.

Chat check: which choices leak information?

- A** Divide every image by the fixed constant 255.
- B** Use the test accuracy to choose the hidden-layer width.
- C** Place near-duplicate images of the same item in training and test.

Chat: select every problematic choice and explain why.

Leakage can make evaluation look too optimistic

B Model selection

Repeated test-based choices adapt the development process to the test set.

C Related examples

Near-duplicates make the test easier than truly new examples.

Fit learned preprocessing on training data. Keep related examples in the same split.

A confusion matrix locates the mistakes

True / Pred.	A	B	C	D	E
A	42	1	1	1	5
B	1	40	1	5	1
C	1	1	38	1	1
D	1	5	1	36	1
E	5	1	1	1	34

Rows: true class
Columns: predicted class

Diagonal: correct predictions
Elsewhere: confusions

Illustrative five-class example with hypothetical counts.

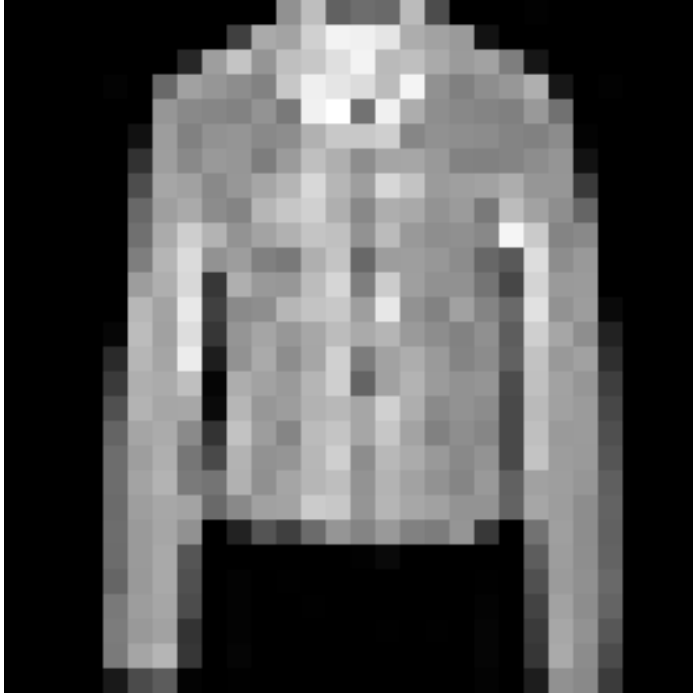
Reading one row gives a class-specific result

True class A predicted as	A	B	C	D	E
Number of examples	42	1	1	1	5

42 correct out of 50 true-A examples: $\text{recall}(A) = 84\%$.

Five true-A examples were classified as E. Inspecting them may explain this specific confusion.

A high score can accompany a wrong prediction



Dataset label: Coat

Illustrative prediction: Pullover
Score: 0.99

A large score reflects the model's ranking. It does not guarantee correctness.

The image is real. The prediction shown here is hypothetical.

Preprocessing must match training and inference

During training

Pixel 204 becomes $204 / 255 = 0.8$

At inference

Sending raw 204 changes the scale the model receives.

The same trained weights can behave very differently with mismatched inputs.

An error suggests an investigation

Observed pattern

Most classes fail suddenly

One pair is often confused

Training improves, validation worsens

Both remain poor

Useful next check

Input scale, shape and class order

Examples, labels and class-specific metrics

Stopping point, capacity and regularization

Data quality, representation and optimization

These are diagnostic starting points. A single symptom does not prove its cause.

Final case: choosing a model for PL2

Candidate	Parameters	Train acc.	Val. acc.	Best epoch
A: baseline	7,850	85.0%	83.0%	12
B: hidden 64	50,890	93.0%	88.0%	8
C: hidden 128	101,770	99.0%	87.0%	5

Chat: choose a candidate and name one missing piece of evidence.

Hypothetical results for discussion. The test set has not been used to choose a candidate.

A defensible choice uses the available evidence

B is the leading accuracy candidate in this example.

Its validation accuracy is highest, with about half the parameters of C.

Confirm repeatability and inspect errors. Evaluate the selected model on the reserved test set, then measure its behaviour on the board.

What we can now explain

Dense layers

Count weights and follow a complete prediction.

Training

Separate loss, gradients and optimizer updates.

Generalization

Read validation curves and investigate errors.

On 24 September: TensorFlow, PyTorch and LiteRT.

Exit question and discussion

**“My model reaches 99% training accuracy.
Why might it still be a poor model?”**

Chat: one reason and one check you would perform.

Questions about the forward pass, training loop or model evaluation

Further reading

[Fashion-MNIST dataset](#)

Images, labels and the original benchmark description.

[TensorFlow: gradients and automatic differentiation](#)

A practical explanation of the forward and backward computations.

[TensorFlow: overfit and underfit](#)

Examples of learning curves and regularization.

[Keras: layers, losses and callbacks](#)

Links are clickable. Additional technical sources appear in the speaker notes.